

The fifteen percent that has nothing to do with getting the math right

Slice books-26 — August 21, 2026

Michael, thank you for the Sage print-out. Uploading your real books instead of letting me theorize was the single most useful thing that could have happened to this slice, and I want to start by telling you what it changed.

I read all eighteen PDFs, checked ninety-one figures by re-deriving each one, and then deleted the thirteen Sage reports the way you asked. Everything they said is written down permanently first — that record is in the repository now, and I will come back to what it found, because some of it needs your attention.

But the reports also answered a question I had been about to ask you, and the answer turned into this slice.

The thing I did not know we were missing

Every piece of payroll code we have built so far answers one question: **how much** comes out of the check. Gross to net, federal withholding, Social Security, Medicare, Paid Leave, WA Cares, L&I. Eight slices of it, tested to the penny.

Not one line of it answered a different question: **by when does that money have to reach the IRS.**

I want to be precise about why that matters, because it is not a small gap. The penalty for depositing late is up to **15%** of the deposit, and it is completely independent of whether the arithmetic was right. You could run a flawless payroll — every paycheck correct to the cent, every rate current, every W-4 applied properly — and still lose 15% of the money because it arrived on a Thursday instead of a Wednesday. The calculation being perfect offers no protection at all.

I checked whether we had anything for this. I searched the entire codebase for the words the IRS uses for this rule, and the only matches were in the cannabis sales-reporting code, which is unrelated. There was nothing. Not a partial implementation, not a stub — nothing.

Which schedule you are on, and how I know without asking you

The IRS puts every employer on one of two schedules. **Monthly** means everything you paid in a calendar month is deposited together by the 15th of the next month. **Semiweekly** means each payday's taxes are due within a few days of that payday.

You are on the **semiweekly** schedule. I did not take your word for it and I did not guess — I proved it twice, two different ways, and the two agree.

The first proof is your own filing. Your Q2 2026 Form 941 came with a **Schedule B** attached, filled in. Only semiweekly depositors file Schedule B; a monthly depositor uses a different box on the 941 itself. So your existing return already declares which schedule you are on, and it says semiweekly.

The second proof is the arithmetic. The IRS adds up line 12 from four quarters of 941s, and \$50,000 is the dividing line. Your Q2 2026 line 12 was **\$14,204.57**. Four quarters at that level is **\$56,818.28**, which is over the line by \$6,818.28 — about 13.6% above it.

Two unrelated methods, same answer. That is the standard I want every number in this system to meet.

The part that would have gone wrong, and the reason it did not

Here is the trap, and I want to walk you through it because it is genuinely subtle and it would have cost you money.

The IRS has a sensible rule for brand-new businesses. If you had no payroll in the lookback period, those quarters count as zero, and zero is under \$50,000, so a new business starts out monthly. Reasonable.

Now think about what this system looks like on **January 1, 2027**. It is brand new. Its tables are empty. There is no payroll history in it, because it has never run a payroll.

So the natural thing — the thing almost any software would do — is look at those empty tables, see no history, and conclude "monthly." And it would be **wrong**, because you are not a new employer. Greenway has been paying employees for years. It is the *software* that is new, and the IRS has never cared what software you use. Depositing monthly when you are required to

deposit semiweekly is not one late deposit. It is a late deposit **every single payday for a year**, each one exposed to that 15%.

So I built it with no default at all. If the system does not have the four quarters of history it needs, it stops and says so, names exactly which quarters are missing, and explains that guessing here risks 15% under IRC 6656. It will not produce an answer it cannot support.

A genuinely new employer's zeros still work — but somebody has to *enter* them. "I have evidence the number was zero" and "I have no evidence" look identical to a computer unless you deliberately keep them apart, and the whole design of this module is keeping them apart.

The lookback period is not last year

This is the part I would want you to remember if you remember nothing else, because it is the one everybody gets wrong, including people who do this for a living.

The four quarters that decide your 2027 schedule are **July 1, 2025 through June 30, 2026**. Not calendar 2026. The period ends eighteen months before the payroll it governs, and the second half of 2026 does not count toward it at all.

That is deliberate on the IRS's part — it means you always know your schedule before the year starts, rather than discovering it partway through. The practical consequence for you is reassuring: **your 2027 schedule was already locked in by the middle of 2026, and nothing that happens in 2027 can change it.**

Two related details worth knowing. The number that counts is line 12 **as originally filed** — if you later amend a quarter with a 941-X, the lookback total does *not* move. The IRS's own example is an employer who reported \$45,000, later found a \$10,000 error, amended it, and still stayed monthly, because the test reads what was originally reported. That is the opposite of an accountant's instinct about corrections, so the system stores the as-filed figure and will not quietly substitute an amended one.

And "semiweekly" does not mean you deposit twice a week. The IRS says so outright: the terms "don't refer to how often your business pays its employees." It names which rulebook applies, not how often you write a check. You pay every other Friday, so you will make **one deposit per payday — about 26 a year**, each due the Wednesday after its Friday.

Your paydays, and the six Thursdays

Since you pay on Fridays, your deposits are due the following Wednesday. Five days later, never the same week.

I checked this against your real Q2 2026 paydays, the seven dates on your Schedule B. All seven are Fridays exactly fourteen days apart — your "every two weeks on Friday," confirmed by arithmetic rather than by my memory of what you told me. The system produces the correct Wednesday for all seven.

Then there is the wrinkle. Federal holidays push deposit due dates out, and the rule is more generous than it first appears: you get at least **three business days** after the period closes, and a holiday *anywhere* in those three days buys you another day — including a holiday that falls *before* the Wednesday you were aiming at.

That distinction matters more than it sounds. The obvious way to write this is "take the following Wednesday, then push it off any holidays." It agrees with the correct answer almost every week of the year, which is exactly what makes it dangerous. The IRS publishes the counterexample: pay on a Friday, with the following Monday a holiday, and the deposit "normally due on Wednesday may be made on Thursday." The Monday holiday is *before* the Wednesday, so the shortcut leaves it on Wednesday and gets it wrong. I wrote it the correct way, counting business days, and there is a test that deliberately breaks it the wrong way to prove the difference is caught.

For 2027, your first payroll year: **six** of the fifty-three Fridays fall due on a Thursday rather than a Wednesday.

- January 15 → due **January 21**
- February 12 → due **February 18**
- May 28 → due **June 3**
- July 2 → due **July 8**
- September 3 → due **September 9**
- October 8 → due **October 14**

One narrow point on holidays, in your favor. For deposit purposes a "legal holiday" means a holiday in the **District of Columbia**, and specifically *not* other state holidays. So DC Emancipation Day (April 16) moves your deposit, and a Washington State holiday does not. A day off in Olympia does not move an IRS deadline.

I did not hardcode a holiday list, because a hardcoded list quietly expires and yours would have expired in the first week of the year that matters. The system generates them from the rules. To prove the rules are right, I made it reproduce the 2026 list the IRS actually prints — all twelve dates and names, including the awkward one, "July 3 — Independence Day (observed)," because July 4, 2026 falls on a Saturday. Getting a published year exactly right is the only real evidence it will get an unpublished year right.

One more rule, dormant but watching

If you ever accumulate **\$100,000 or more** of taxes on a single day, that money is due the **next business day**, whatever schedule you are on.

You are nowhere near this — your largest single-day liability in Q2 2026 was \$2,500.65, and your entire quarter was \$14,204.57. But I built the check anyway rather than deciding you are too small, for a specific reason. You told me:

“*I pay myself once at the end of the year.*”

A single large year-end owner payment is precisely the shape of event that trips this rule. It is dormant, not absent, and the system watches for it.

There is a sting in the tail worth knowing: a *monthly* depositor who trips this becomes semiweekly the next day and stays semiweekly for the rest of that year and all of the following one. It does not apply to you, since you are already semiweekly, but the system explains it either way.

What your Sage reports told me about your live books

Now the part that needs your attention. I found **four things** in your current books that look wrong. All four are arithmetic, all four are written up with the numbers, and I want to be careful about which are *findings* and which are *questions*.

1. The EAF surcharge is being charged at the wrong rate for at least one employee. Sage's own Tax Liability Report contradicts itself — it shows the same levy at both 0.03000 and 0.64000. Your Q2 ledger shows \$66.73 of EAF where the return you actually filed shows \$20.68, a gap of \$46.05. For the year, \$134.04 charged against \$51.39 correct — about **\$82.65 overstated**. One employee's wages explain \$57.33 of that, so at least one other employee is affected too. Your *filed return was correct*; it is your ledger that disagrees with it.

2. Your unemployment rate might be wrong, and this one is a question, not a finding. Your EAMS portal says your 2026 UI rate is **0.37%**. Your filed 5208A and Sage both used **0.64%**. If the portal is right, Q2 was overpaid by \$186.09, which annualizes to roughly **\$462.50**. The likeliest explanation is a stale tax table in Sage — the portal screenshot is dated August 19 and the return was prepared August 21 — but I could not *prove* that, so I am not asserting it. This one needs you to check which figure ESD actually assigned you.

3. WA Cares is being withheld but reported as zero. Sage withheld **\$399.76** in Q2 — I verified that is exactly 0.58% of \$68,923.45, paycheck by paycheck — and your Paid Leave return reports WA Cares wages of 0.00, premiums of 0.00, and zero for all ten employees, with the exemption column blank for every one of them. Money left your employees' checks and the return says it did not.

4. The Paid Leave small-employer exemption may be unclaimed. The exemption box is unchecked and you are paying an employer share of \$222.51 a quarter — roughly **\$890 a year** — with ten employees.

A few other things I confirmed while I was in there: nobody is on direct deposit (sixteen paper checks in June); **you are absorbing your employees' L&I share** rather than withholding it, about \$1,331 a year out of your pocket; sick leave is accruing at exactly one hour per forty hours *worked*, excluding sick hours, verified on all eight paychecks, with four employees carrying advanced negative balances; and Sage has **three** salaried people, where you told me "hourly for all employees, salary for me."

That last one is not a defect, just a mismatch between what you told me and what Sage holds, and I would rather ask than assume.

What I need from you

Seven questions are written up in the repository record. The ones that touch real money, in the order I would ask them:

- 1. The unemployment rate — 0.37% or 0.64%?** This is roughly \$462 a year and it also affects what the system uses going forward.

2. **WA Cares reported as zero** while being withheld. Do you know why the return shows nothing?
3. **The Paid Leave employer share** — around \$890 a year. Should the small-employer exemption be claimed?
4. **Three salaried employees** in Sage (Jim and Theresa Becker among them). Is that right, or should they be hourly?
5. I **kept** the five `example_form_*` PDFs, because you described those separately as reference material rather than as part of the Sage print-out. Tell me if you want those gone too and they are gone.

Still outstanding from earlier slices, and I will keep listing these until they are closed: your exact S-election tax year, ending AAA from Schedule M-2, the \$6699(e) per-shareholder amount, the quarterly federal short-term rates, your depreciation schedule and asset list, the work papers, twelve years of returns, the intercompany detail, and the Wells Fargo loan details.

How hard I leaned on this

You asked me to keep the pace slow and test it and then test it some more, so here is the accounting.

Eighty-eight new tests, plus 193 checks built into the module itself. The full suite is **7,653 tests across 368 files**, all passing. Types clean, lint clean with no new warnings.

Twelve new legal citations, every one quoted word for word from IRS Publication 15 and mechanically verified against a stored copy of the publication before I wrote a line of code. That verification caught me once: I had trimmed a parenthetical out of the middle of the \$100,000 rule, which turns a quotation into a paraphrase wearing a quotation's clothes. Put back.

Then the part I think is actually worth the most. I deliberately broke the finished code **ten** different ways — one at a time — to confirm the tests would notice. Threshold comparison flipped. Lookback period off by a year. The nudged Wednesday instead of counting business days. Holidays ignored entirely. The semiweekly week shifted a day. Monthly due dates in the wrong month. Date parsing made lenient. And the dangerous one: missing history silently defaulting to monthly.

Ten introduced, ten caught, none survived. Then I confirmed the file was restored exactly as it was.

I also found one real bug during the work, and it was in my own test rather than in the code: I built a date as `2026-08-0` plus a number, which produces `2026-08-010` — not a real date. The engine refused it, correctly. Strict date parsing caught a mistake that lenient parsing would have silently rolled over into September. I fixed the test and added another one to pin that refusal in place.

The fixtures throughout are your real filed numbers and the IRS's own worked examples, not figures I invented. So if this ever breaks, it means the code stopped agreeing with a return you have already filed with the government.

The safety net had a hole in it, and I found it by accident

This one is not about payroll, but you should know about it, because for six rounds of work it meant less protection than you were told you had.

Every time I finish a slice, I run the whole test suite on my machine, and there is also an automatic check that runs the same tests up on GitHub after the work is submitted. Two independent looks at the same question. That automatic check is the one that is supposed to catch me if I ever break something and do not notice.

I ran my tests, saw all 7,653 of them pass, and told you it was green. Then I went and looked at the automatic check, and it was **red**. Not because of my work — it had been red on every single round since books-20. **Six rounds of work were accepted while that check was failing**, including the one I finished just before this.

Here is the part that actually cost you something. The automatic check does two jobs in order: first it inspects the code for style problems, then it runs the tests. It stopped at the first job. Which means **the tests never ran up there at all** — not for books-20, 21, 22, 23, or 25. Every time I reported "all tests pass," that was true on my machine and confirmed by nothing else. The second pair of eyes had been closed the whole time.

The cause was almost insultingly small. In two of the test files, three places loaded a standard tool using older syntax the project's style rules forbid. Each one had a comment attached meant to say "allow this here" — but the comment named the **wrong rule**. So it excused nothing, and then the checker also complained about the pointless comment. Three small spots, six complaints, and the whole safety net switched off.

Before I touched it, I proved it was not my doing, because "it was already broken" is exactly what someone says when they broke it. I checked out a clean copy of the project as it stood *without* any of my work, ran the identical command, and got the identical failure and the identical error count. My work had added nothing to it.

Then I fixed it the way twenty other files in that same folder already do it, and proved the fix was real rather than just quiet.

Those three spots do something important: they read the actual program files off the disk to confirm a fact about them. If a read like that silently comes back empty, the test still *passes* — it just is not checking anything anymore. So I deliberately pointed each of the three at a filename that does not exist, to make sure each one would notice and fail. All three failed loudly, exactly as they should. Then I put them back and confirmed the files were byte-for-byte identical to before.

The automatic check is now green — the first time since books-20 — and I confirmed from its own log that the tests genuinely ran this time, all 368 files including the 88 new ones from this slice.

I have written this down as a permanent rule for myself, number 57: a green run on my machine is not a green safety net, and I do not get to quietly accept a red one just because it was red before I arrived. Six rounds had taught this project that red was simply the normal colour of that light. That is precisely how a real break walks in without anyone noticing.

Where this sits

You now have the piece that knows *when* the money is due. The next slices build the reports and the forms on top of it: books-27 is the reports engine and the 941-to-W-2 reconciliation, books-28 the federal forms including that Schedule B you are now required to file, books-29 the Washington forms, and books-30 the filing exports.

One inherited duty worth flagging now, since it shapes books-28: because you are semiweekly, your 941 is incomplete without Schedule B, and Schedule B reports liability **day by day**. That means the daily figure has to be captured as payroll runs, not reconstructed from a quarterly total at filing time. The system is now set up to do that.

Nothing in here was guessed. Where I could not prove something — the unemployment rate being the clearest case — I have written it down as a question instead of an answer.